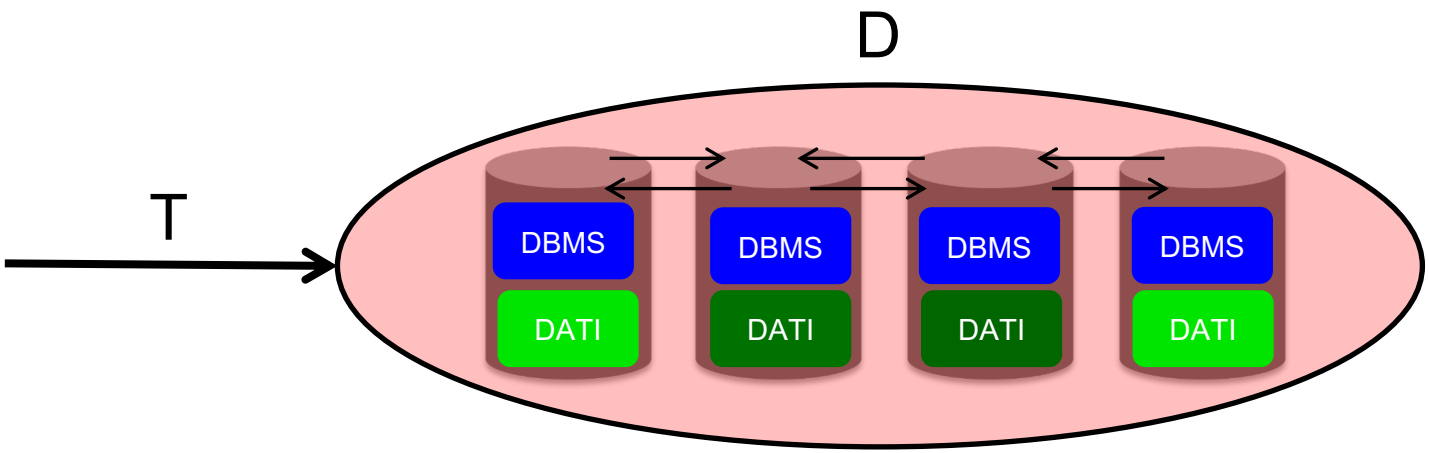
Gestione delle transazioni nelle basi di dati distribuite

Paolo Atzeni, Stefano Ceri, Luigi Bellomarini

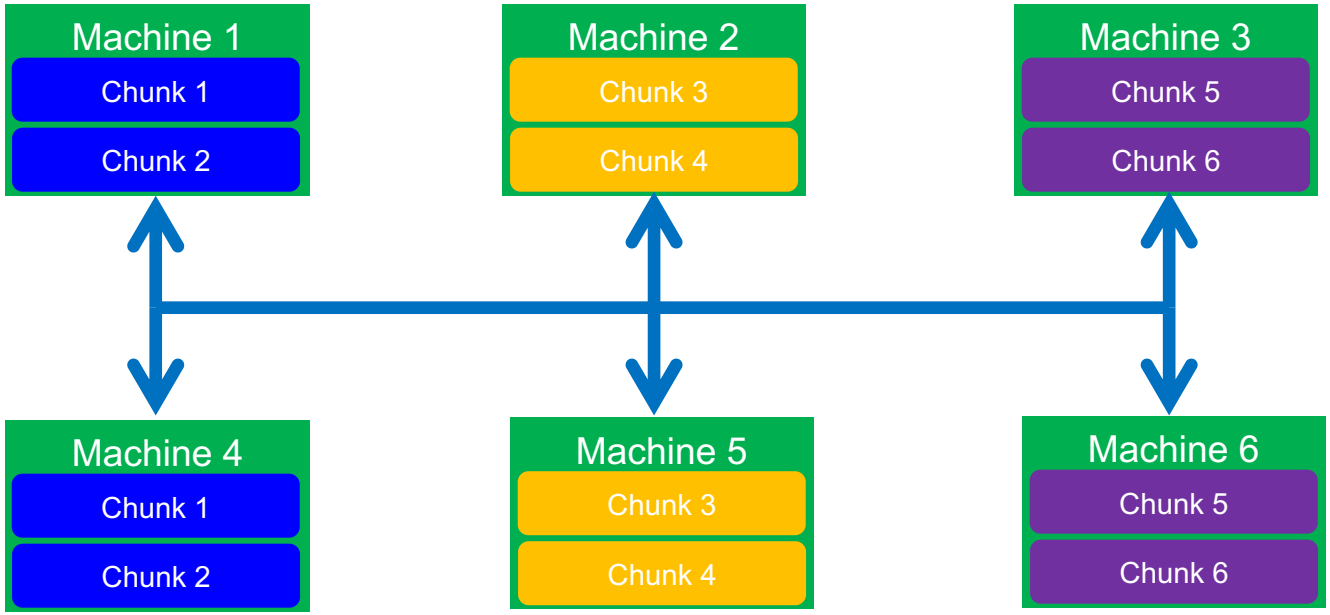
Base di dati e transazioni distribuite

- I dati e il controllo (i processi del DBMS) sono distribuiti su server (nodi) diversi, coordinati fra loro
- Le operazioni sono per quanto possibile delegate ai vari nodi



Architetture per basi di dati distribuite

- **Partizionamento** (*sharding*): suddivisione dei dati fra vari nodi
- **Replicazione**: stessi dati su più nodi
- Partizionamento e replicazione possono essere combinati



Transazioni remote e distribuite (1)

- Transazione **remota**
  - Interagire con una base di dati nel contesto di un'altra

```
UPDATE scott.dept@sales.us.americas.acme_auto.com
  SET loc = 'NEW YORK'
  WHERE deptno = 10;
UPDATE scott.emp@sales.us.americas.acme_auto.com
  SET deptno = 11
  WHERE deptno = 10;
COMMIT;
```

- Transazione **distribuita** su più basi di dati
  - Interagire con più basi di dati contemporaneamente

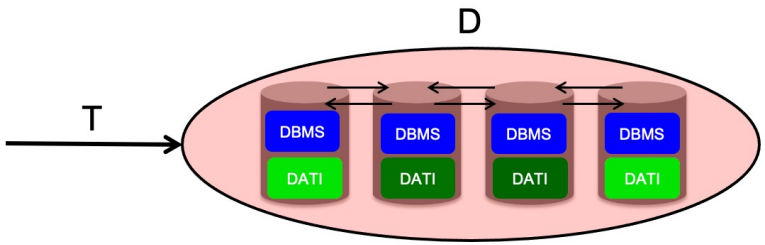
```
UPDATE scott.dept@sales.us.americas.acme_auto.com
  SET loc = 'NEW YORK'
  WHERE deptno = 10;
UPDATE scott.emp
  SET deptno = 11
  WHERE deptno = 10;
COMMIT;
```

- http://pages.di.unipi.it/ghelli/didattica/bdldoc/B19306_01/server.102/b14231/ds_concepts.htm

Transazioni remote e distribuite (2)

- Transazione **distribuita** su una base di dati distribuita
  - Interagire con una base di dati distribuita
  - La distribuzione è spesso trasparente per l'utilizzatore

```
UPDATE scott.dept
  SET loc = 'NEW YORK'
  WHERE deptno = 10;
UPDATE scott.emp
  SET deptno = 11
  WHERE deptno = 10;
COMMIT;
```



- http://pages.di.unipi.it/ghelli/didattica/bdldoc/B19306_01/server.102/b14231/ds_concepts.htm

Tecnologia delle transazioni distribuite

- Ci concentreremo sulle **basi di dati distribuite** che usano solo **sharding**
  - Approfondimenti sulla replicazione nella prossima lezione
- Con solo *sharding*, la distribuzione non influenza consistenza e durabilità
  - La **consistenza** (nell'accezione fino ad ora studiata) non dipende dalla distribuzione, i vincoli di integrità sono locali:
    - la tecnologia DBMS attuale non gestisce vincoli distribuiti
  - La **durabilità** non dipende dalla distribuzione: ogni sistema garantisce la durabilità locale utilizzando meccanismi di recupero locali
    - log, checkpoint e dump
- Aspetti rilevanti per:
  - Concorrenza (**isolamento**)
  - Affidabilità (**atomicità**)
  - Ottimizzazione delle interrogazioni (**efficienza**):
    - non approfondiamo, anche se sarebbe interessante

Controllo di concorrenza

- In teoria:
  - **La serializzabilità locale non è sufficiente per la serializzabilità globale.** Esempio, due transazioni t_1 e t_2 su due nodi A e B

    $t_1 : r_{1A}(x)\ w_{1A}(x)\ r_{1B}(y)\ w_{1B}(y)$

    $t_2 : r_{2B}(y)\ w_{2B}(y)\ r_{2A}(x)\ w_{2A}(x)$

    Sui nodi

    A: $r_{1A}(x)\ w_{1A}(x)\ r_{2A}(x)\ w_{2A}(x)$

    B: $r_{2B}(y)\ w_{2B}(y)\ r_{1B}(y)\ w_{1B}(y)$
- Però:
  - se i lock in scrittura sono mantenuti fino al commit, che è **globale** (cioè coinvolge tutti i nodi) la serializzabilità è garantita dall'ordinamento dei commit e dalle letture delle versioni appropriate
- Il tutto, come sempre, si potrebbe approfondire, ma ci accontentiamo

Guasti nei sistemi distribuiti

- **Guasti sui singoli nodi (locali):** come nel caso centralizzato
- **Perdite di messaggi:** lasciano i protocolli in stato di incertezza
  - I messaggi sono seguiti da conferme (ack)
  - Si possono perdere gli uni come gli altri
- **Partizionamento della rete**

Protocollo di commit a due fasi

- Una decisione (**commit** o **abort**) per tutti i partecipanti ad una transazione
- Simile ad un contratto:
  - Due o più parti si impegnano e una terza parte (il mediatore o il notaio) ratifica
  - I **partecipanti**, detti anche **resource manager** (RM)
  - Il coordinatore (che può essere uno dei partecipanti), detto anche **transaction manager** (TM)

Esempi

- **esercizio 5 del 21/02/2017** (organizzazione di un viaggio)
- **esercizio 3 del 16/07/2007** (Napoleone in battaglia)

Protocollo di commit a due fasi (2)

- Due fasi, **preparazione** e **conferma**, separate da una decisione
  - preparazione:
    - "siete disponibili al commit?"
  - decisione
    - se tutti d'accordo "ok" altrimenti "lasciamo perdere";
  - conferma:
    - comunicazione a tutti, con conferma di ricezione
- Il tutto con scambio (rapido) di messaggi fra coordinatore e partecipanti con scritture nei log e con conferme (ack)

Record nel log del coordinatore

- **`prepare`**: contiene l'identificatore della transazione e dei partecipanti
  - serve a ricordare di avere chiesto disponibilità
- **`global commit`** o **`global abort`**
  - decisione
- **`complete`**
  - conclusione del protocollo

Record nei log dei partecipanti

- record scritti nel log come già visto per la gestione tradizionale: **`begin`**, **`insert`**, **`delete`**, e **`update`** oltre a **`commit`** e **`abort`**
- **`ready`** conferma di avere dato disponibilità (**irrevocabile**) a partecipare al commit; contiene riferimenti alla transazione e al coordinatore
  - dopo aver scritto **`ready`** il partecipante perde la propria autonomia decisionale sulla transazione
  - viene scritto dopo avere verificato che, dal punto di vista del partecipante, la transazione può andare in commit (ha le risorse, soddisfa i vincoli, …);

Prima fase del protocollo

- Il coordinatore scrive **`prepare`** nel log e **poi** invia un messaggio di **`prepare`** ai partecipanti. Fissa un timeout per le risposte.
- Ogni singolo partecipante che riceve il messaggio di **`prepare`**
  - se è in grado di partecipare al commit, scrive nel proprio log un record di **`ready`** e trasmette al coordinatore un messaggio di **`ready`** per comunicare la propria disponibilità
  - altrimenti, può inviare un mesaggio di **`not-ready`** (oppure niente)
- Il coordinatore riceve risposte fino al timeout
  - se ha ricevuto tutte le risposte e sono tutte positive, scrive un record **`global commit`** nel proprio log
  - altrimenti (almeno una risposta mancante o negativa) scrive un record di **`global abort`**

Seconda fase del protocollo

- Il coordinatore trasmette la decisione (globale) ai partecipanti che hanno risposto **`ready`** e a quelli la cui risposta non è arrivata, fissando un nuovo time-out
- Ogni partecipante che riceve il messaggio, scrive la decisione (record di **`commit`** o **`abort)`** nel proprio log, invia un *ack* al coordinatore e implementa la decisione con le opportune scritture
- Il coordinatore riceve i messaggi di *ack* dai partecipanti
  - se allo scadere del time-out ne manca qualcuno (fra coloro che avevano risposto **`ready`** o non avevano risposto), invia loro di nuovo la decisione
  - quando tutti hanno risposto scrive un record di **`complete`** nel log

The two-phase commit protocol

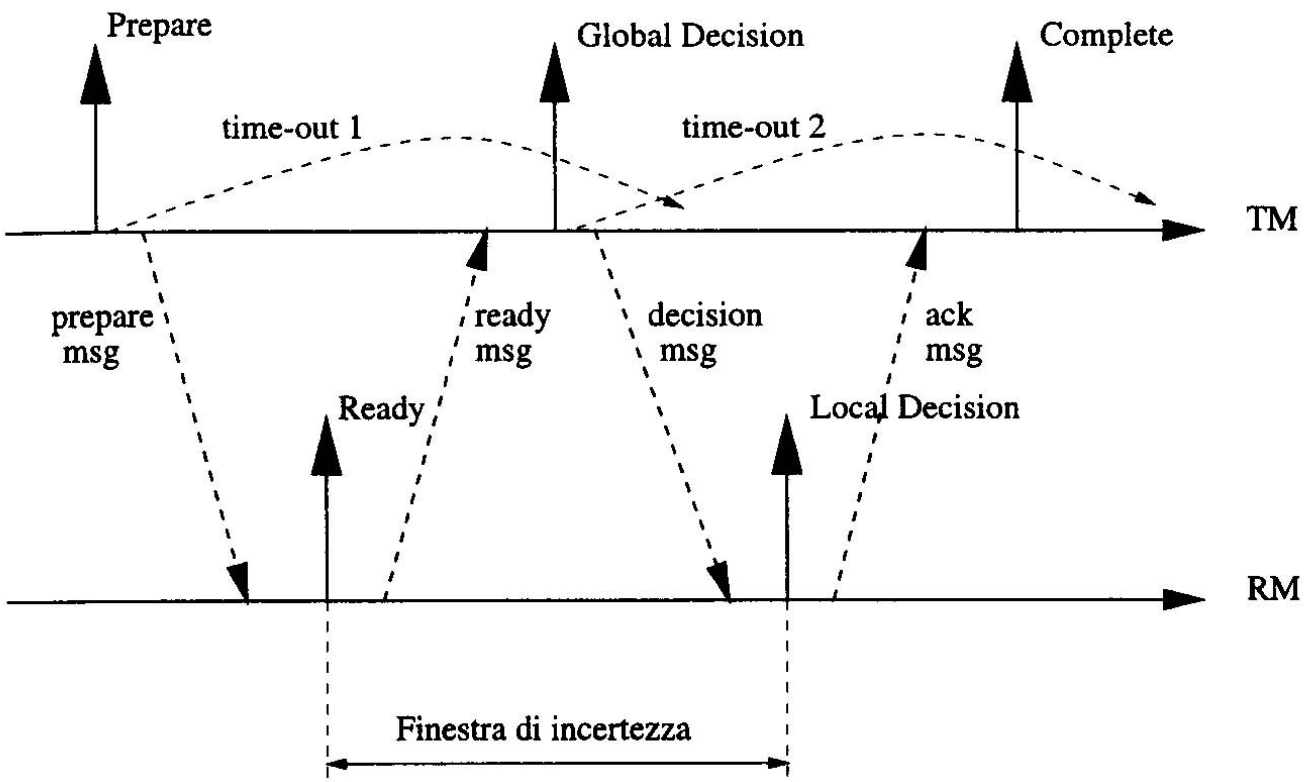


Figura 10.9 Protocollo di commit a due fasi

2PC dal punto di vista di una transazione

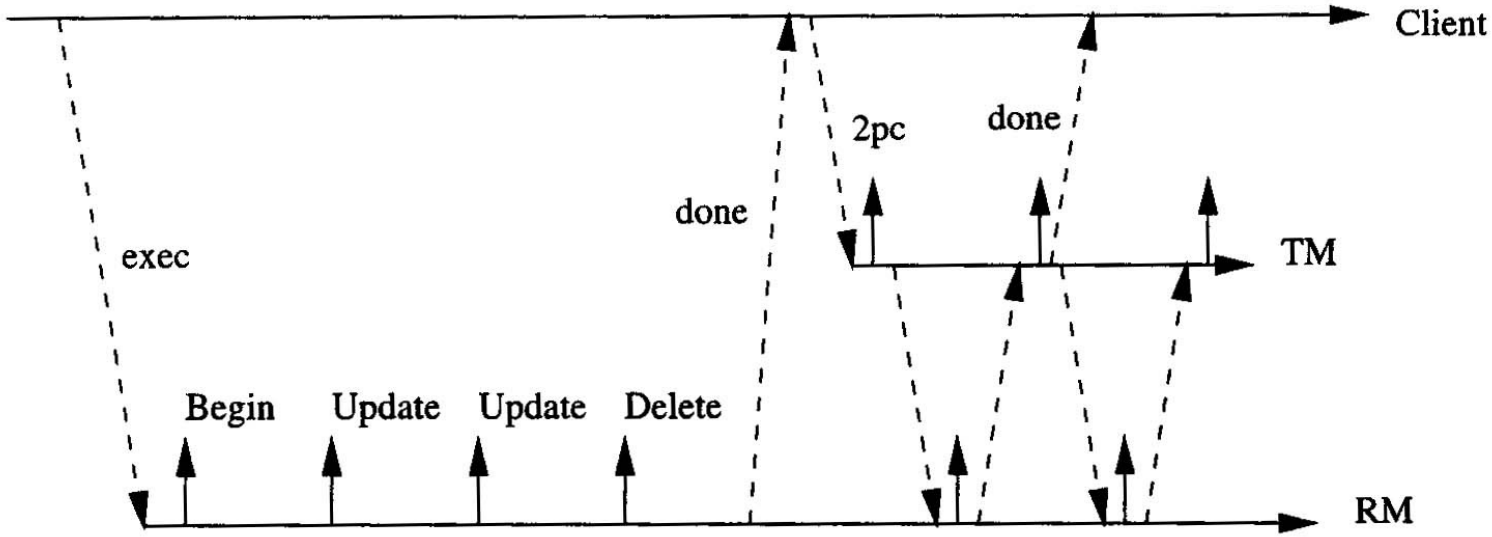


Figura 10.10 Protocollo di commit a due fasi nel contesto di una transazione

Incertezza e ripristino

- Un partecipante in uno stato di **`ready`**
  - ha perso la propria autonomia
  - attende la decisione del coordinatore
  - è lasciato in uno stato incerto in caso di fallimento del coordinatore; se ha acquisito lock (e questo accade quasi sempre) le risorse relative sono bloccate
- **Finestra di incertezza**
  - l'intervallo tra la scrittura del **`ready`** e la ricezione della decisione
  - il partecipante resta in attesa della decisione e non può fare niente

Recovery (ripristino) di un partecipante

- Per ogni transazione
  - se l'ultimo record nel log relativo ad essa è:
    - un **`commit`**, le sue azioni debbono essere confermate (*redo*);
    - un'azione o un **`abort`**, allora la transazione deve abortire, con *undo* delle sue azioni
    - un **`ready`**, il partecipante non ha informazioni sull'esito della transazione e ha bisogno di notizie dal coordinatore:
      - può aspettare o chiederle

Recovery (ripristino) del coordinatore

- Per ogni transazione
  - se l'ultimo record è
    - **`complete`**, non è necessario fare niente (la decisione è stata presa e tutti sono stati informati e hanno recepito l'informazione)
    - una decisione, allora la decisione è stata presa e non può essere cambiata, ma non è sicuro che tutti siano informati e allora va ripetuta la seconda fase
    - **`prepare`**, ci possono essere partecipanti in attesa della decisione
      - la soluzione più semplice è **`global abort`**
      - In alternativa, è possibile ripetere la prima fase

Guasti

- Crash, perdite di messaggi (di notifica o di ack), partizionamenti sono gestiti tutti nello stesso modo: se non si riceve conferma si considera fallita la fase:
  - se è la prima, (di solito) si abortisce
  - se è la seconda, si ripete la seconda fase (la decisione è stata presa)
- Nota bene:
  - la decisione è presa fra la prima e la seconda fase, quindi la seconda fase non può modificarla

Rivediamo gli esempi

- **esercizio 5 del 21/02/2017** (organizzazione di un viaggio)
- **esercizio 3 del 16/07/2007** (Napoleone in battaglia)

Esercizi

- **Esercizi 6 e 7 del 21/06/2018**